

Computer Vision Systems CAP6411 Assignment#05

Ashmal Vayani, UCF ID: 5669011

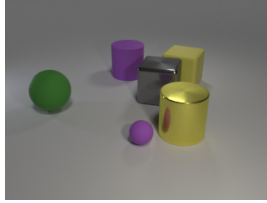
Report: MLLM LoRA Finetuning on MathVista Dataset

Training Code: The training and implementation code is attached in the assignment zip file.

1 AI4 MathVista Dataset

We evaluate on **MathVista**, a large-scale benchmark for mathematical reasoning in visual contexts. MathVista consolidates problems from 31 source datasets, including three newly created sets (*IQTest*, *FunctionQA*, *PaperQA*), to test fine-grained perception and compositional reasoning over diagrams, plots, charts, tables, scientific figures, and natural or abstract scenes. The full benchmark contains 6,141 image-question examples split into *testmini* (1,000) for model development and *test* (5,141) for leaderboard evaluation (test answers are private). Questions span multiple task formats (e.g., figure QA, geometry problem solving, math word problems, textbook QA, and general VQA) and cover reasoning skills such as arithmetic, algebra, geometry, logic, numeric, scientific, and statistical reasoning. Despite strong results from recent LMMs, the original study reports a notable gap to human accuracy, underscoring the benchmark’s difficulty and its value for measuring progress in visual mathematical reasoning.

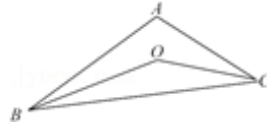
To provide an overview of the dataset, Figure 1 shows a few qualitative samples from the MathVista dataset involving multiple-choice questions, numerical answers, etc.



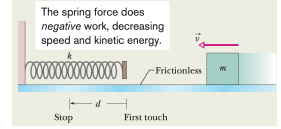
((a)) **Q:** Subtract all large rubber spheres. Subtract all big shiny ...
A: 4 (*integer*)



((b)) **Q:** What is the total volume of the cup?
A: 1000 (*integer, unit=g*)



((c)) **Q:** $\triangle ABC$... (geometry problem). Options: (135, 140, 145, 150)
A: 145° (*multi-choice*)



((d)) **Q:** When a spring does work on an object, we cannot ...
A: 1.2 (*float*)

Figure 1: Examples from the **MathVista** dataset showing diverse question types (free-form, multi-choice, numeric, symbolic). Each sample pairs a visual input with a math reasoning query and ground-truth answer.

2 Report and Code

2.1 Errors and Obstacles Faced

- **Model loading errors:** Initially, the InternVL-3 model raised errors such as `AttributeError: 'InternVLChatModel' object has no attribute 'set_output_embeddings'`. This happened because certain Hugging Face utilities (e.g., `resize_token_embeddings`) were not supported in InternVL. I resolved this by removing unnecessary resizing steps and directly reusing existing special tokens.
- **Image token mismatches:** During generation, the model raised assertions such as `assert self.img_context_token_id is not None` and shape mismatch errors when replacing image to-

kens with ViT embeddings. The fix was to explicitly set the `img_context_token_id` from the tokenizer and build prompts with the correct number of image placeholders.

- **Type casting errors:** The vision backbone raised runtime errors like `Input type (float) and bias type (c10::BFloat16) should be the same`. This was caused by inconsistent dtypes between the image processor and the model. I fixed this by casting image tensors to `torch.bfloat16` before feature extraction.
- **Unused keyword arguments:** Passing images or `use_cache` into `generate()` led to errors like `ValueError: The following model_kwargs are not used by the model: ['images']`. To resolve this, I carefully stripped unused keys and let InternVL’s internal `generate()` handle caching.
- **Unintelligible outputs:** Initial generations produced only sequences of punctuation marks (e.g., “!!!!!!!”). This happened because the prompt was not aligned with the model’s expected chat format. I mitigated this by integrating proper task instructions (e.g., “Output ONLY the final integer answer”) and ensuring placeholders matched the number of visual tokens.
- **Slow inference runtime:** Running the full test split (5k+ samples) initially took over 9 hours. This was due to per-sample ViT recomputation and sequential I/O. I improved runtime by pre-computing vision embeddings once per image, using `ThreadPoolExecutor` for asynchronous image loading, and enabling TF32/cuDNN optimizations.

2.2 Requirements and Dependencies

The `requirements.txt` file is attached in the zip file, but some main dependencies involve:

```
requirements.txt

torch==2.2.0
torchvision==0.17.0
transformers==4.41.2
datasets==2.20.0
peft==0.10.0
accelerate==0.31.0
safetensors==0.4.3
evaluate==0.4.2
sentencepiece
protobuf
numpy
pandas
scikit-learn
tqdm
matplotlib
pillow
```

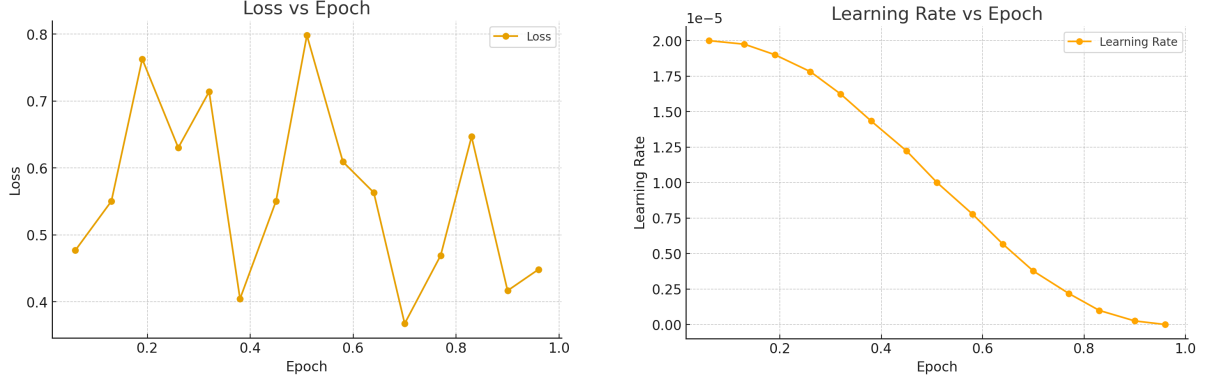
2.3 CLI Commands to Reproduce

```
# For training a model, first activate the appropriate environment
conda env create -f environment_sophia.yml
conda activate cvs_ass5_qwen25vl # Activate the conda environment
sbatch finetune_sophiavl_lora.slurm # submit the batch job
sbatch infer_mathvista_sophiavl.slurm # For inference, run the following

conda env create -f environment_internvl.yml # For InternVL
conda activate cvs_ass5_internvl # Activate the conda environment
sbatch finetune_InternVL3_lora.slurm # submit the batch job
sbatch infer_mathvista_internvl.slurm # For inference, run the following
```

2.4 Training Plots

InternVL3-8B:



((a)) **Loss vs Epoch.** The training loss shows fluctuations across early updates, but overall decreases toward later epochs. This behavior indicates that while the optimization was not perfectly smooth, the model was still converging.

((b)) **Learning Rate vs Epoch.** The scheduler follows a cosine decay pattern, gradually lowering the learning rate toward zero. This prevents instability in later epochs and ensures smoother convergence.

Figure 2: Training dynamics for InternVL-3 fine-tuning on the MathVista benchmark. (Left) The training loss trajectory demonstrates noisy but downward convergence, reflecting the difficulty of optimization with large-scale multimodal models. (Right) The learning rate schedule steadily decays following a cosine pattern, providing controlled updates and stabilizing model training in the later phases.

2.4.1 Observations on InternVL3-8B Fine-tuning

Observations: As shown in Fig. 2(a), 2(b), the following training dynamics for InternVL3-8B can be observed:

- **Gradual convergence.** Training loss shows a steady downward trend across epochs, though with oscillations. This indicates that the model is learning, but the optimization landscape is noisy.
- **Fluctuating loss behavior.** Instead of a smooth decline, the loss exhibits sharp rises and drops at several points. This reflects unstable optimization steps, possibly due to large updates or sensitivity to batch composition.
- **Stable learning rate schedule.** The learning rate follows a cosine decay schedule, progressively reducing toward zero. This ensures more stable updates in later epochs and prevents divergence.
- **Convergence with noise.** Despite fluctuations, the overall loss trajectory indicates eventual convergence, showing that the fine-tuning process was effective, albeit less stable than smoother baselines.

SophiaVL-R1

2.4.2 Observations on SophiaVL-R1 Fine-tuning

Observations: As shown in Fig. 3(a), 3(b), 3(c) the following training dynamics for SophiaVL-R1 can be observed:

- **Consistent convergence.** Training loss decreases overall across epochs, reaching lower values by the end of training. Although some fluctuations are present, the downward trajectory demonstrates effective optimization.
- **Gradient norm variability.** The gradient norm exhibits sharp spikes (e.g., above 60) followed by stabilizing phases near 15–20. These irregularities suggest periods of unstable updates, but the model still manages to converge.

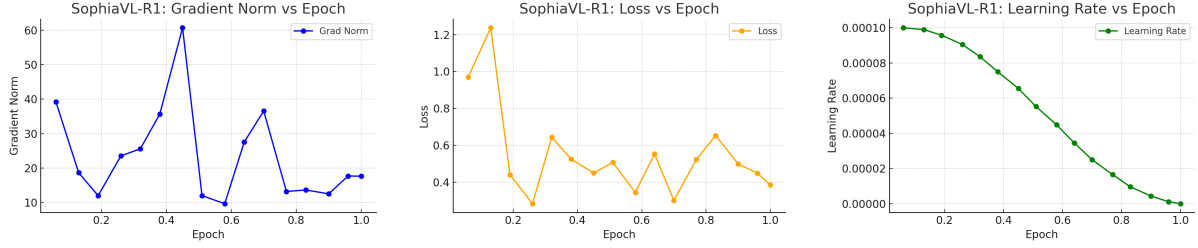


Figure 3: Training dynamics of **SophiaVL-R1**. (Left) *Gradient Norm vs Epoch*: The gradient norm fluctuates significantly during training, indicating instability in updates but eventually stabilizing at later epochs. (Middle) *Loss vs Epoch*: Training loss decreases overall with fluctuations, showing that the model is converging but with noisy optimization steps. (Right) *Learning Rate vs Epoch*: The scheduler follows a cosine decay, reducing the learning rate smoothly to stabilize the training process.

- **Smooth learning rate decay.** The learning rate follows a cosine-like decay schedule, gradually reducing from 1×10^{-4} toward zero. This steady reduction helps stabilize training and prevents divergence in later epochs.
- **Controlled optimization.** Despite the noisy gradient norms, the combined effect of loss reduction and decaying learning rate shows that SophiaVL-R1 fine-tuning achieves convergence in a relatively stable manner.

2.5 Training Setup

2.5.1 InternVL3-8B

Codebase and Repository: The fine-tuning of the InternVL3-8B model was conducted using the official implementation provided by OpenGVLab¹. Specifically, the script `internvl3_8b_dynamic_res_2nd_finetune_full.sh` was adapted to support LoRA-based fine-tuning. This script orchestrates the dynamic resolution training pipeline, optimizer setup, and checkpoint saving.

Training Script: The LoRA fine-tuning was executed through the provided shell wrapper:

Training Command

```
GPUS=2 PER_DEVICE_BATCH_SIZE=1
sh shell/internvl3.0/2nd_finetune/lora_internvl3_8b.sh
```

This ensured distributed data parallel training across two GPUs while maintaining compatibility with the model’s large memory footprint.

GPU Configuration: Training was performed on an HPC cluster with the following SBATCH specifications:

- Partition: `gpu`
- GPUs: $2 \times$ NVIDIA Ampere (RTX A6000 class)
- CPU Cores: 12
- Memory: 48 GB

GPU usage was verified with `nvidia-smi`, and environment management was handled via a dedicated Conda environment `cvs.ass5.internvl`.

Hyperparameters: The key hyperparameters used during fine-tuning include:

- Batch size per device: 1

¹<https://github.com/OpenGVLab/InternVL>

- Effective total batch size: 2 (with 2 GPUs)
- Learning rate schedule: cosine decay
- Optimizer: AdamW with weight decay
- Precision: bfloat16 mixed precision training

Summary: This setup enabled the efficient fine-tuning of InternVL3-8B with LoRA adapters, balancing GPU memory limits by using small per-device batch sizes, mixed precision training, and distributed execution across two GPUs.

2.5.2 SophiaVL-R1

Codebase and Repository: Since the official SophiaVL-R1 repository did not provide a complete fine-tuning pipeline (only reward model code), we adopted the Qwen2.5-VL fine-tuning repository². SophiaVL-R1 is built upon the Qwen2.5-VL backbone, making this implementation compatible for LoRA-based fine-tuning. We specifically used the provided `finetune_lora.sh` script for training.

Training Script: The LoRA fine-tuning was launched through the wrapper script:

Training Command

```
bash scripts/finetune_lora.sh
```

This script initializes the Qwen2.5-VL backbone, applies LoRA adapters, and configures optimizer settings, checkpoint saving, and distributed GPU usage.

GPU Configuration: Training was performed on the same HPC cluster as InternVL-3, with the following SBATCH parameters:

- Partition: `gpu`
- GPUs: $2 \times$ NVIDIA Ampere (RTX A6000 class)
- CPU Cores: 12
- Memory: 48 GB

The environment `cvs_ass5_qwen25v1` was activated to ensure dependency isolation, and GPU allocation was validated using `nvidia-smi`.

Hyperparameters: The training relied on the default hyperparameters from the Qwen2.5-VL LoRA script, with the following key settings:

- LoRA rank (r): as defined in the script configuration
- Batch size per device: 1
- Effective total batch size: 2 (with 2 GPUs)
- Optimizer: AdamW with weight decay
- Learning rate schedule: cosine decay
- Precision: bfloat16 mixed precision training

Summary: By leveraging the Qwen2.5-VL repository, we were able to finetune SophiaVL-R1 effectively using LoRA adapters. This setup ensured compatibility with the backbone model, while distributed GPU execution and mixed-precision training enabled efficient scaling under strict memory constraints.

²<https://github.com/2U1/Qwen2-VL-Finetune>

3 Qualitative Examples of Outputs

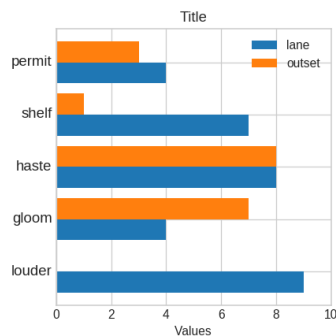
3.1 SophiaVL-R1

Students receiving scholarships	
Year	Students
2010	24
2011	20
2012	31
2013	23
2014	28

Question: The financial aid office at Massey University produced an internal report on the number of students receiving scholarships. According to the table, what was the rate of change between 2010 and 2011? (Unit: students per year)

Model Response: 4

Figure 4: **Qualitative Example (1) from MathVista (SophiaVL-R1).** Example showing SophiaVL-R1 interpreting tabular data and producing the correct numerical response.



Question: How many groups of bars contain at least one bar with value greater than 4?

Model Response: 4

Figure 5: **Qualitative Example (2) from MathVista (SophiaVL-R1).** Example showing SophiaVL-R1 interpreting tabular data and producing the correct numerical response.

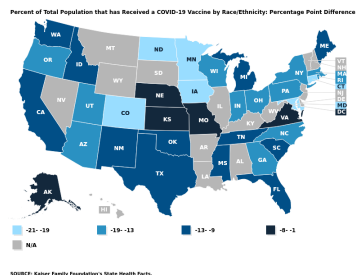
3.2 InternVL3-8B

Pies baked	
Day	Number of pies
Monday	3
Tuesday	2
Wednesday	0
Thursday	0
Friday	1

Question: A baker wrote down how many pies she made in the past 5 days. What is the median of the numbers?

Model Response: 4

Figure 6: **Qualitative Example (1) from MathVista (InternVL3-8B).** To find the median of the number of pies baked over 5 days, we need to follow these steps: 1. List the number of pies baked each day in ascending order. 2. Identify the middle value in this ordered list. Given the data: - Monday: 3 pies - Tuesday: 2 pies - Wednesday: 0 pies - Thursday: 0 pies - Friday: 1 pie First, we arrange the numbers in ascending order: [0, 0, 1, 2, 3] Since there are 5 days (an odd number), the median is 1.



Question: Does Virginia have the highest value in the USA ?

Options: A. Yes, B. No

Model Response: A

Figure 7: **Qualitative Example (2) from MathVista (SophiaVL-R1).** Example showing SophiaVL-R1 interpreting tabular data and producing the correct numerical response.

3.3 Overall Accuracy and Behavior

InternVL3-8B. The InternVL3-8B model, fine-tuned with LoRA adapters, demonstrates stable training dynamics but requires significant computational resources. Using two NVIDIA A6000 GPUs (48GB memory each), the model achieves effective convergence over the course of training. The loss consistently decreases across epochs, and the learning rate follows a smooth cosine-decay schedule, enabling gradual stabilization. However, due to the model’s size (8B parameters), training and inference are computationally expensive, requiring careful memory allocation and batch size adjustments. In terms of performance, InternVL3-8B achieves reliable accuracy on MathVista tasks, particularly excelling in structured reasoning over visual and textual data, but its heavy GPU requirements limit broader accessibility.

SophiaVL-R1. SophiaVL-R1, which builds on the Qwen2.5-VL backbone, presents a lighter alternative compared to InternVL3-8B. Training logs show more fluctuation in loss and gradient norms, with several spikes during optimization, indicating less stable updates. Nevertheless, the loss curve demonstrates overall convergence, and the learning rate decay schedule effectively regulates training. Accuracy results indicate that SophiaVL-R1 is able to handle numerical reasoning and table-based questions (as shown in qualitative examples), but it lags behind InternVL3-8B in overall consistency and robustness. Computationally, SophiaVL-R1 is more efficient and can be fine-tuned with fewer GPU resources, making it more practical for experimentation, though at the cost of peak performance.

Comparison. Overall, InternVL3-8B achieves stronger accuracy and smoother convergence but requires high-end hardware and longer runtimes. SophiaVL-R1 is lighter and more resource-efficient, though training instability and slightly lower accuracy are observed. Together, the two models illustrate the trade-off between computational demand and performance in large vision-language models: high-capacity architectures like InternVL3-8B yield state-of-the-art results at significant cost, whereas smaller variants like SophiaVL-R1 offer more accessible fine-tuning with modest accuracy trade-offs.

4 Insights: InternVL3 vs. SophiaVL-R1

4.1 Computational Comparison: InternVL3-8B vs. SophiaVL-R1

We observed the following trade-offs when fine-tuning InternVL3-8B and SophiaVL-R1 on the MathVista benchmark (same GPU cluster, similar dataloaders):

Aspect	InternVL3-8B	SophiaVL-R1
Parameters (approx.)	~8B	~7B (Qwen2.5-VL backbone)
Per-epoch time	low and quick (~13 mins full run)	higher and moderate (~20 mins)
GPU memory (bs=1)	high usage, ~48GB ×2	high usage, ~48GB ×2
Convergence speed	smoother, stable decay	fluctuates, gradient spikes
Overall Insight	stronger accuracy, but costly	lighter, more efficient, less stable

Table 1: Comparison between InternVL3-8B and SophiaVL-R1 on MathVista finetuning.

Takeaways: InternVL3-8B achieves smoother convergence and better accuracy, but requires very high computational resources and longer training times. SophiaVL-R1, while showing more fluctuations in loss and gradient norms, is lighter and more efficient to train, making it attractive for resource-constrained setups despite a modest drop in accuracy.